Homework Assignment 6

This document contains the write-ups for the three challenges of **Assignment 6**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	Nuclear Pasta	`fsuCTF{sp4ghett1_c0d3_15_a_d3fens3_mech4n1sm}`	- **Identify Encryption:** Analyze the binary to find a "spaghetti code" convert function, which is actually a simple XOR cipher using the key `0xBB`. - **Extract Ciphertext:** Use a debugger like GDB to extract the target encrypted bytes (`ct`) from memory. - **Decrypt:** Perform a bitwise XOR operation between each extracted byte and `0xBB` to reveal the flag.
Problem 2	Old School Gauntlet	`fsuCTF{b3tt3r_7h4n_cry5t41_4rm0r}`	- **Step 1:** Set a breakpoint after the Parent Process check and overwrite the buffer at `$rbp-0x120` with a null byte to hide GDB. - **Step 2:** Reach the Ptrace check and use the jump command to force execution to the success address at `main+1237`. - **Step 3:** Reach the Time check and use jump to skip to `main+1358`, bypassing the 1-second limit. - **Step 4:** Inspect the memory at `$rbp-0x250` to read the decrypted flag.
Problem 3	Upper Management	`fsuCTF{5ti11_b3tter_7h4n_73ch_supp0r7}`	- **Determine Length:** Use `ltrace` to find that the program requires an input length of exactly 38 characters. - **Symbolic Execution:** Because the transformation function is too complex to reverse by hand, use the angr library in Python to define the input as symbolic variables. - **Configure the solver** to find a path that reaches the "Good enough" success message while avoiding the "Wrong" failure path.

PROBLEM 1 - Nuclear Pasta

In this challenge, we are given a single Linux binary called `nuclear_pasta`.

1. Analysis

The first thing I did was check what kind of file I was dealing with using the `file` command.

```
└─$ file nuclear_pasta
nuclear_pasta: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-li
nux-x86-64.so.2, BuildID[sha1]=c3892e9c8b095876847334f3da9cea6d160aa73b, for GNU/Linux 4.4.0, not stripped
```

The fact that it is **not stripped** will make reading the code easier sinces the developer left the function and variable names inside the binary.

I gave the file execution permissions (`chmod +x nuclear_pasta`) and ran it to see how it behaves:

```
└─$ ./nuclear_pasta
Gimme your best plate of pasta o' flag:
test
Yuck, that's not the right flag at all.
```

Apparently, the program just checks the flag. I enter a string, and it tells I'm wrong.

I ran `strings` to see if the flag was just hidden in the text. I didn't find the flag, just the success/failure messages.

```
u3UH
Gimme your best plate of pasta o' flag:
%127s
Mmmm... that's a tasty flag
Yuck, that's not the right flag at all.
;*3$"
GCC: (GNU) 15.2.1 20260103
```

I uploaded the file to **Dogbolt.org**. I checked different outputs, but **Hex-Rays** gave the most readable code. I found two important functions: `main` and `convert`.

- Inside `main`, I saw this:

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  unsigned __int8 *i; // [rsp+28h] [rbp-128h]
  char v5[128]; // [rsp+30h] [rbp-120h] BYREF
  char s1[136]; // [rsp+B0h] [rbp-A0h] BYREF
  unsigned __int64 v7; // [rsp+138h] [rbp-18h]

  v7 = __readfsqword(0x28u);  1
  puts("Gimme your best plate of pasta o' flag:");
  __isoc23_scanf("%127s", v5);
  for ( i = (unsigned __int8 *)v5; *i; ++i )  2
    s1[i - (unsigned __int8 *)v5] = convert(*i);
  if ( !strcmp(s1, &ct) )  3
    puts("Mmmm... that's a tasty flag");
  else
    puts("Yuck, that's not the right flag at all.");
  return 0;
}
```

  1. It asks for my input.
  2. It uses a loop to change every letter I type using the `convert` function.
  3. It compares my modified input with a variable called `ct` using `strcmp`.
- I then looked at the `convert` function:

```
__int64 __fastcall convert(unsigned __int8 a1)
{
  int v3; // [rsp+8h] [rbp-Ch]
  int v4; // [rsp+Ch] [rbp-8h]
  int i; // [rsp+10h] [rbp-4h]

  v3 = 255;
  v4 = 40;
  do
  {
    v4 ^= v3;
    v3 = 0;
    for ( i = 0; i <= 17; ++i )
    {
      v4 += 2;
      v3 -= 6;
    }
    if ( a1 )
      a1 ^= 0xBBu;
    else
      a1 = 0;
  }
  while ( !(v4 % v3) );
  return a1;
}
```

I realized the `do-while` loop is a distraction. The condition `!(v4 % v3)` is only true once. The only important part is `a1 ^= 0xBB`. This means the "spaghetti code" is just a simple **XOR cipher** with the number **0xBB**.

2. Solution Strategy

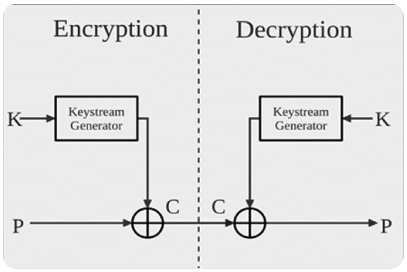
I tried to use `strings` to find the secret `ct`, but it was not there. This is because `ct` is not a "string" in the code, but a list of bytes in the memory.

To get those bytes, I used **GDB**. I knew the variable was named `ct` because the program was not stripped.

```
gef➤  x/48xb &ct
0x4040 <ct>:    0xdd    0xc8    0xce    0xf8    0xef    0xfd    0xc0    0xc8
0x4048 <ct+8>:  0xcb    0x8f    0xdc    0xd3    0xde    0xcf    0xcf    0x8a
0x4050 <ct+16>: 0xe4    0xd8    0x8b    0xdf    0x88    0xe4    0x8a    0x8e
0x4058 <ct+24>: 0xe4    0xda    0xe4    0xdf    0x88    0xdd    0xde    0xd5
0x4060 <ct+32>: 0xc8    0x88    0xe4    0xd6    0xde    0xd8    0xd3    0x8f
0x4068 <ct+40>: 0xd5    0x8a    0xc8    0xd6    0xc6    0x00    0x00    0x00
```

> The command `x/48xb` allows us to retrive 48 bytes in hex

Now I have the "encrypted" bytes from the memory and I know the secret key is `0xBB`. The theory of XOR is: if `P ^ Key = C`, then `C ^ Key = P`. So, I just need to take every byte from `ct` and do XOR with `0xBB`.



3. Execution and Flag

Before doing everything, I wanted to be sure that my XOR theory was correct. I took the first four bytes from the `ct` variable in GDB and calculated them manually:

- 0xdd ⊕ 0xbb = `0x66` ('f')
- 0xc8 ⊕ 0xbb = `0x73` ('s')
- 0xce ⊕ 0xbb = `0x75` ('u')
- 0xf8 ⊕ 0xbb = `0x43` ('C')

The result is the start of the flag format so I knew I was on the right path.

I wrote a Python script to decrypt all the bytes I found in GDB:

```
ct_bytes = [
    0xdd, 0xc8, 0xce, 0xf8, 0xef, 0xfd, 0xc0, 0xc8, 0xcb, 0x8f, 0xdc, 0xd3, 0xde, 0xcf, 0xcf,
    0x8a, 0xe4, 0xd8, 0x8b, 0xdf, 0x88, 0xe4, 0x8a, 0x8e, 0xe4, 0xda, 0xe4, 0xdf, 0x88, 0xdd,
    0xde, 0xd5, 0xc8, 0x88, 0xe4, 0xd6, 0xde, 0xd8, 0xd3, 0x8f, 0xd5, 0x8a, 0xc8, 0xd6, 0xc6
]

key = 0xbb
flag = ""

for b in ct_bytes:
    flag += chr(b ^ key)

print(f"Flag: {flag}")
```

I executed the script and it returned the flag.

Flag: `fsuCTF{sp4ghett1_c0d3_15_a_d3fens3_mech4n1sm}`

PROBLEM 2 - Old School Gauntlet

In this challenge we are provided with an ELF binary executable named `gauntlet`. When executed normally, it displays a welcome message and immediately begins a series of "checks" before finishing.

```
└─$ ./gauntlet
Welcome, Warrior of the Amlodd!
Prove your might in... the gauntlet...
Testing for guardians...
Passed!
Testing your tracers...
Passed!
Testing your time...
This program is fine :)
I assume you are mighty. But are you wise?
I am going to decrypt the flag at 0x7fff414f6d40, but it will only reveal itself to those who are worthy.
Done.
```

The program claims to decrypt a flag at a specific memory address, but it finishes execution without actually printing it.

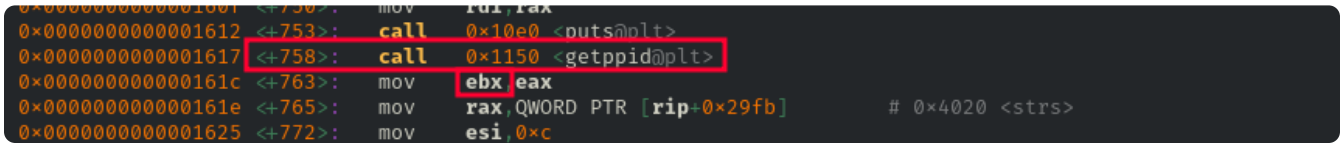
1. Analysis

To understand the internal logic, I loaded the binary into GDB (`gdb -q ./gauntlet`) and examined the assembly code of the `main` function using `disas main`.

I observed three distinct blocks of code that functioned as protections or checks:

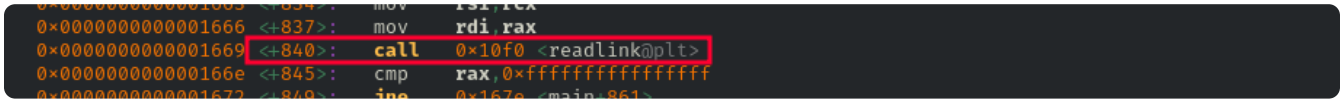
1. **Parent Process Check:**
   I observed calls to `getppid@plt` followed by `readlink@plt`.

Calls `getppid` and stores the result in `ebx`

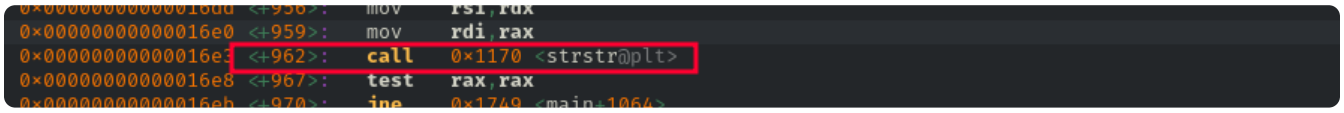


Builds the string `/proc/[PID]/exe`

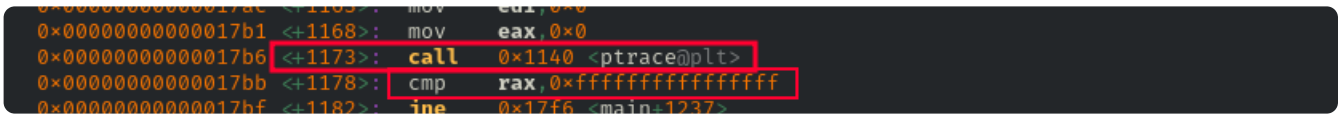
The program then reads the symbolic link `/proc/[PID]/exe`.



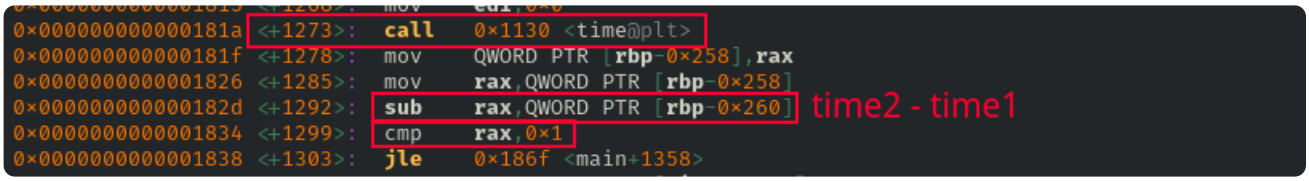
Later, I saw a call to `strstr@plt`, which suggests the program then scans the parent process name for specific forbidden strings.



2. **Debugger Check (Ptrace):**
   I found a call to `ptrace@plt`. Immediately after, the code compares the result (`rax`) with `-1` (`0xffffffffffffffff`).

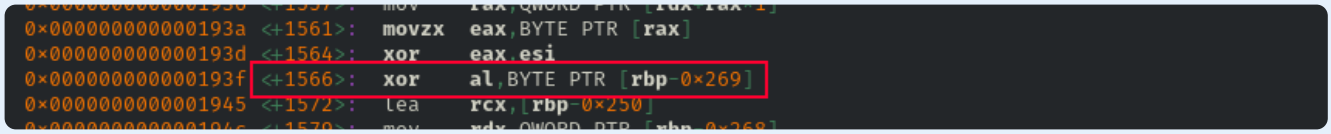


   If `ptrace` returns -1, it means the process is already being traced by a debugger. A conditional jump (`jne`) directs the flow to an exit routine if this check fails.
3. **Execution Time Check:**
   I observed two calls to `time@plt` at different points in the execution. Between them, the code performs a subtraction (`sub`) and compares the result against `0x1` (1 second).
   If the execution takes longer than 1 second, the program jumps to a failure block.



If any of these checks fail, the program skips the decryption logic or terminates.

> ⓘ **Integrity Variable**
>
> Additionaly, a local variable at `[rbp-0x269]` serves as a cumulative XOR key for the final flag decryption. This variable is only updated if the previous checks are passed successfully:
>
> 1. **1:** Increments by `1` after the Parent Check.
> 2. **2:** Bitwise left-shifts by 4 (`shl 4`) after the Ptrace Check.
> 3. **3:** Bitwise left-shifts by 2 (`shl 2`) after the Time Check.
>
> *The final flag is decrypted using this variable. If the checks are bypassed using manual patching (like changing jumps to `NOP`), the variable never reaches the required value of 64, which results in an incorrectly decrypted flag.*



2. Solution Strategy

I will use **Dynamic Analysis** within GDB to bypass these checks at runtime.

My plan is to:

1. **Identify the critical decision points** (jumps) for each of the three protections.
2. **Manipulate the memory or registers** at runtime to trick the program into believing it is running normally, "bypassing" the gauntlet while keeping the decryption logic intact.

3. Execution and Flag

I started the program with `start`. This pauses execution at the `main` function.

```
gef➤ start
```

Step 1: Bypassing the Parent Check

The first check reads the parent process name. Even if the system call succeeds, the buffer will contain ".../gdb", which triggers a detection mechanism later (a `strstr` search). To bypass this, I set a breakpoint at the instruction immediately *after* the read is performed (`*main+861`).

```
gef➤ break *main+861
gef➤ continue
```

At this point, the buffer at `$rbp-0x120` contained the string ".../gdb". I wrote a `0` (null byte) to the beginning of this address. This makes the program read the string as empty.

```
gef➤ set {char}($rbp-0x120) = 0
```

Step 2: Breakpoints

With the first check neutralized, I set breakpoints at the remaining critical decision points (Ptrace and Time) and the finish line *(if I don't put the last one the program will finish and the RAM will be wiped before we can retrieve the flag)*.

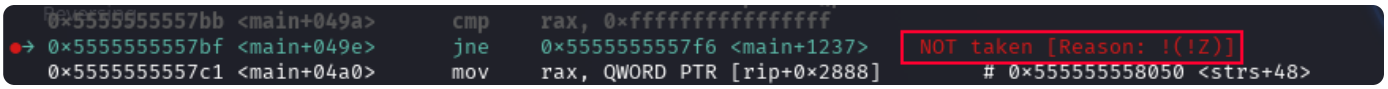
```
gef➤ break *main+1182
gef➤ break *main+1303
gef➤ break *main+1647
```

Step 3: Bypassing the Ptrace Check*

I continued execution until the Ptrace check.

```
gef➤ continue
```

`[ main+1182 ]` - GDB indicated the jump was `NOT TAKEN` (failure).



I used the `jump` command to skip to the success address (`*main+1237`).

```
gef➤ jump *main+1237
```

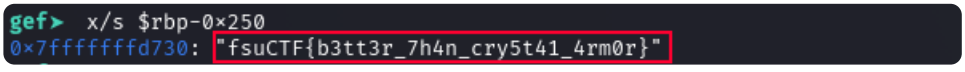
Step 4: Bypassing the Time Check

`[ main+1303 ]` - Since I was manually stepping through the code, the execution time was much longer than the allowed 1 second, so the jump was `NOT TAKEN`. I forced the jump to the success address (`*main+1358`).


```
gef➤ jump *main+1358
```

Step 5: Decryption

`[ main+1647 ]` - Having successfully bypassed all three gates (Guardian, Ptrace, and Time), I inspected the stack memory at `$rbp-0x250` where the flag was constructed.



Flag: `fsuCTF{b3tt3r_7h4n_cry5t41_4rm0r}`

PROBLEM 3 - Upper Management

In this challenge, we are presented with a Linux binary named `upper_management`. The prompt mentions a 3rd quarter IPO and a management team looking for a "brand new flag" to show off to investors.

1. Analysis

I started running `file` to understand what kind of file I was dealing with.

```
└─$ file upper_management
upper_management: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-lin
ux-x86-64.so.2, BuildID[sha1]=b87b4e358f381bb12543ba380d1c5c410755eed9, for GNU/Linux 3.2.0, stripped
```

The word **"stripped"** means the developers removed all the function and variable names, so I won't see a clear `main()` function or helpful labels in the debugger.

Next, I checked for hardcoded strings using `String` but it didn't give any relevant information. I then tried running the binary and giving it a dummy input, and nothing interesting happened.

```
└─$ ./upper_management
Management needs you to produce a flag.
I don't know what the spec is supposed to be, I'll just let you know if it looks wrong.
test
Wrong. Next.
```

I tried using `ltrace` to see if the program used `strcmp` to compare my input against the real flag in memory.

```
└─$ ltrace ./upper_management
puts("Management needs you to produce "...Management needs you to produce a flag.
I don't know what the spec is supposed to be, I'll just let you know if it looks wrong.
)                                = 128
__isoc99_scanf(0x402098, 0x7ffd0074e7c0, 0x7effd5417790, 0x7effd5417790test
) = 1
strlen("test")                                                    = 4
puts("Wrong. Next."Wrong. Next.
)                                                 = 13
+++ exited (status 1) +++
```

I noticed that it calls `strlen` on my input but **never reaches a comparison function**. This suggested that the program first checks if the input length is correct. If it isn't, it just exits.

Since I couldn't see the symbols, I used **Dogbolt.org** to look at the decompiled code from Ghidra. I searched for the "Management" strings to find the main logic. I found a function that contained the following:

```
puts(
    "Management needs you to produce a flag.\nI don\'t know what the spec is supposed to be, I\'ll just let
    );
__isoc99_scanf("%127s",local_98);
sVar3 = strlen(local_98);
if (sVar3 == 0x26) {
  for (local_d0 = local_98; *local_d0 != '\0'; local_d0 = local_d0 + 1) {
    cVar1 = FUN_004011d6((int)*local_d0,(int)local_d0 - (int)local_98);
    *local_d0 = cVar1;
  }
  iVar2 = strcmp(local_98,local_c8);
  if (iVar2 == 0) {
    puts("Good enough, ship it.");
    uVar4 = 0;
    goto LAB_00401c23;
  }
}
puts("Wrong. Next.");
```

1. The input must be exactly **38 characters** long (in hex).
2. The program iterates through each character and passes it to a function `FUN_004011d6` along with its index.
3. Looking into `FUN_004011d6`, it was a massive chain of XORs, additions, and shifts that seemed impossible to reverse by hand.

```
char FUN_004011d6(char param_1,char param_2)

{
  if (('/' < param_1) && (param_1 < '~')) {
    return ((((((((((((((((((((((((((((((param_1 + (param_2 + '\x03') * '\x13' ^
                                        (param_2 + '\x02') * '7' ^ param_2 + 2U) +
                                      (param_2 + '\x05') * '\x03' ^ (param_2 + '\x03') * '!' ^
                                     (param_2 + '\x02') * -0xb) + (param_2 + '\x05') * 'O' ^
                                    (param_2 + '\x03') * -0x5e) + param_2 * -0x71 + 0x8e ^
                                    (param_2 + '\x05') * -0x44 ^ (param_2 + '\x01') * 's') +
                                   (param_2 + '\x02') * -0x28 ^ (param_2 + '\x04') * '\x10') +
                                  (param_2 + '\x05') * '\x12' ^ (param_2 + '\x05') * -0x35 ^
                                  (param_2 + '\x02') * -0x18 ^ (param_2 + '\x01') * -0xf) +
                                 param_2 * '?' + 0xc ^ (param_2 + '\x02') * '\b' ^
                                (param_2 + '\x03') * ' ') + param_2 * -0x6b + 0x84 ^
                               (param_2 + '\x02') * -0x2e) + param_2 * -0x1c + 0x1e ^
                               (param_2 + '\x02') * -0x49 ^ (param_2 + '\x03') * '\x0f' ^
                              (param_2 + '\x01') * 'm') + param_2 * -5) - 10 ^
                             (param_2 + '\x02') * -0x77 ^ (param_2 + '\x02') * '9') + param_2 * 'j') -
                        10 ^ (param_2 + '\x01') * -0x4c ^ (param_2 + '\x02') * ']') + param_2 * 'G'
                      + 0xac ^ (param_2 + '\x01') * ';') + param_2 * '\\' + 0x16 ^
                     (param_2 + '\x02') * -0x66 ^ (param_2 + '\x03') * 'U') +
                    (param_2 + '\x04') * -8 ^ (param_2 + '\x01') * '\x05') +
                   (param_2 + '\x05') * 'C' ^ (param_2 + '\x04') * -0x52 ^
                   (param_2 + '\x01') * '\n' ^ (param_2 + '\x01') * '\\' ^
                   (param_2 + '\x01') * '\x11' ^ (param_2 + '\x05') * -0x14 ^
                  (param_2 + '\x05') * -0x2c) + (param_2 + '\x01') * 'F' ^
                  (param_2 + '\x04') * 'c' ^ (param_2 + '\x01') * -0x38) +
                 (param_2 + '\x04') * '\x1d' ^ (param_2 + '\x02') * 'B' ^
                (param_2 + '\x01') * -0x38) + param_2 * '\x03' + 0x44 ^ (param_2 + '\x03') * -0x1c
               ) + (param_2 + '\x01') * '\x19' ^ (param_2 + '\x05') * '\v') +
              (param_2 + '\x05') * ' ' ^ (param_2 + '\x02') * '\x1c') + (param_2 + '\x01') * -0xe
             ^ (param_2 + '\x03') * -0x2b) + param_2 * 'H' + 0x96 ^ (param_2 + '\x05') * 'm') +
            (param_2 + '\x02') * '\f' ^ (param_2 + '\x01') * 'I') + param_2 * '\x14' + 0x2c ^
           (param_2 + '\x04') * -6 ^ (param_2 + '\x01') * '\t') + param_2 * -0x3c + -0x1c;
  }
  puts("Wrong. Next.");
                    // WARNING: Subroutine does not return
  exit(1);
}
```

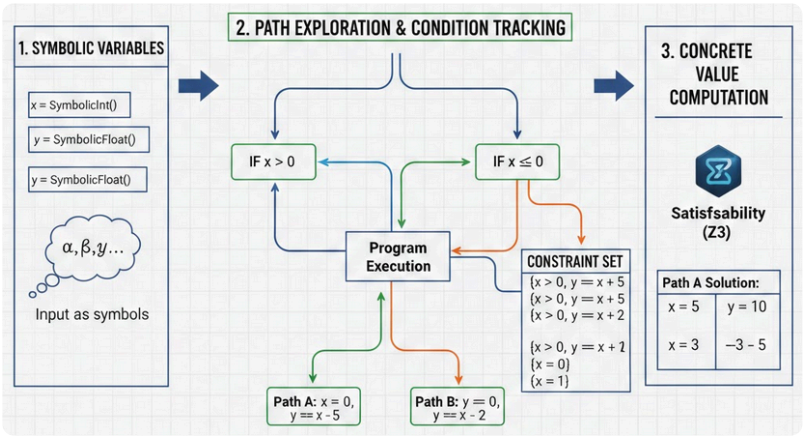
4. Finally, the transformed string is compared against a hardcoded byte array (`local_c8`).

2. Solution Strategy

I could have tried to rewrite the math function in Python and brute-force it character by character, but since I have access to Symbolic Execution tools like **angr**, I decided to use them instead.

What I am going to do is let angr analyze the program and automatically calculate the correct input. Instead of testing many possible values manually, I define the input as symbolic data and tell angr to find a path that reaches the `"Good enough"` message.

> Angr does not execute the program with real numbers at first. It treats the input as symbolic variables and tracks the conditions the program applies to them. Then it computes concrete values that satisfy all those conditions and make the program follow the desired execution path.

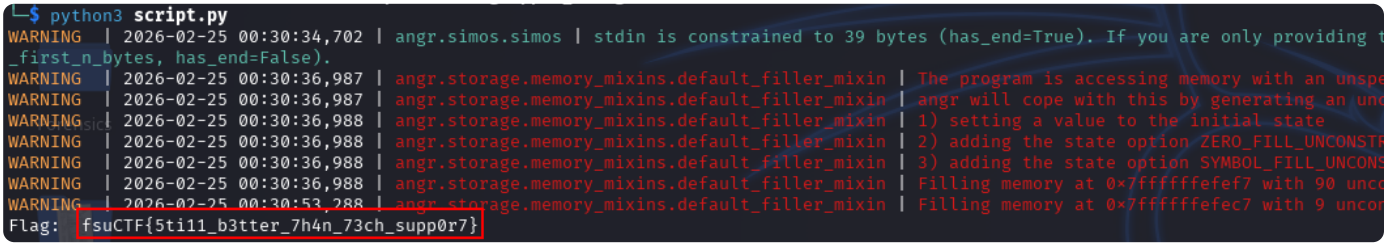


3. Execution and Flag

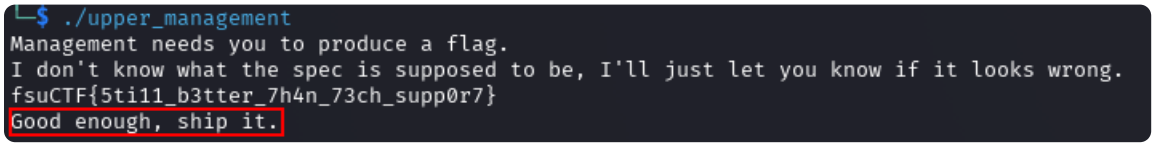
I wrote a short Python script using the `angr` library. I defined the input as a symbolic variable of 38 bytes and told the solver to find a path that prints the success message while avoiding the failure message.

```
import angr
import claripy

project = angr.Project('./upper_management')

flag_chars = [claripy.BVS(f'flag_{i}', 8) for i in range(38)]
flag = claripy.Concat(*flag_chars + [claripy.BVV(b'\n')])

state = project.factory.entry_state(stdin=flag)

for c in flag_chars:
    state.solver.add(c > 32, c < 127)

simgr = project.factory.simulation_manager(state)
simgr.explore(find=lambda s: b"Good enough" in s.posix.dumps(1),
              avoid=lambda s: b"Wrong" in s.posix.dumps(1))

if simgr.found:
    print("Flag:", simgr.found[0].posix.dumps(0).decode())
```

Running the script took about 15 seconds. The solver successfully navigated through all the XORs and additions to find the original characters.



I verified it by running the binary manually:



Flag: `fsuCTF{5ti11_b3tter_7h4n_73ch_supp0r7}`